05

© 한상훈 NestJS 백엔드 마스터

NestJS 백엔드 마스터

PART 1

비동기 I/O의 밑바닥: 스레드 풀, epoll, io_uring

제1부 기반: 백엔드와 Node.js 런타임 · 2장 이벤트 루프와 비동기 I/O

이벤트 루프가 기다리지 않는 동안 기다리는 쪽

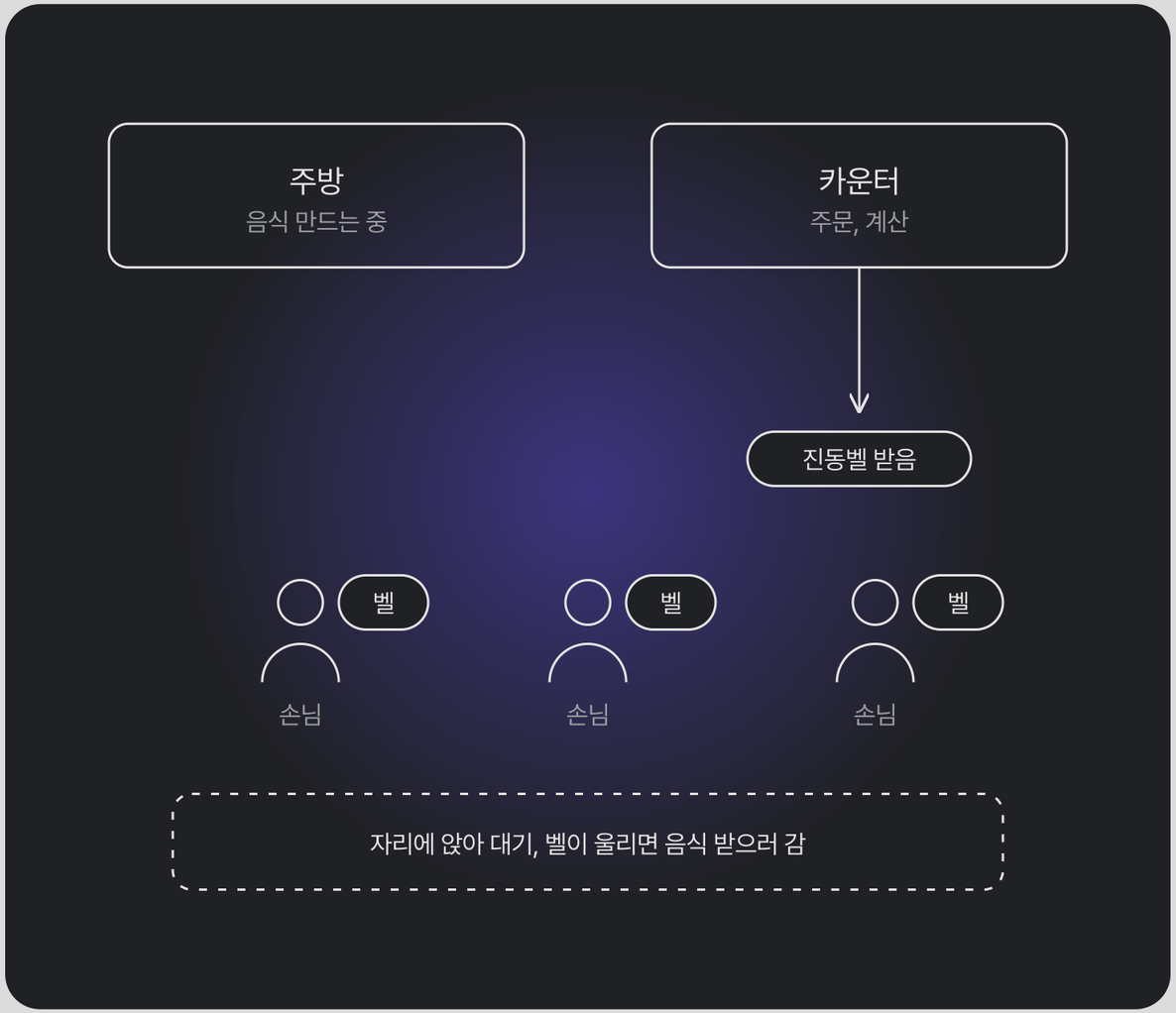
© 한상훈 NestJS 백엔드 마스터

식당 진동벨

→ 주문, 계산 → 진동벨 받고 자리로

→ 카운터 앞에서 기다리지 않음

→ 벨이 울리면 음식 받으러 감



© 한상훈 NestJS 백엔드 마스터

벨은 누가 울리나

→ 손님: 이벤트 루프 (요청 맡기고 다음 일)

→ 진동벨: 콜백, 프로미스

→ 벨을 울리는 사람: ?

→ 벨 누르는 사람이 바쁘면 → 음식이 나와도 벨이 안 울림

→ 답 없는 결제사를 몇 분씩 기다린 `fetch`: 기다린 건 열린 소켓 하나와 운영체제

© 한상훈 NestJS 백엔드 마스터

libuv

→ Node 안에서 이벤트 루프와 비동기 I/O를 맡는 C 라이브러리

→ 자바스크립트 코드 → Node API → libuv → 운영체제

→ 이벤트 루프의 칸(timers, poll, check)도 libuv



© 한상훈 NestJS 백엔드 마스터

기다리는 두 가지 방법

방법	맡기는 말	쓰는 곳
운영체제 알림	"준비되면 알려 줘"	소켓 (HTTP, DB 연결)
스레드 풀	"대신 해 줘"	파일, DNS 조회, 암호 계산, 압축

© 한상훈 NestJS 백엔드 마스터

비동기 I/O

→ 비동기 I/O: I/O를 시켜 두고, 끝났다는 알림을 나중에 받는 방식

→ 시키는 쪽: 이벤트 루프

→ 실제로 기다리는 쪽: 운영체제 커널, 또는 스레드 풀의 스레드

→ 스레드 하나라는 말: 자바스크립트를 실행하는 스레드가 하나라는 뜻

예

`await readFile` → libuv → 풀 대기 줄 → 스레드가 읽기 → 루프에 알림 → poll 칸에서 다음 줄 실행

© 한상훈 NestJS 백엔드 마스터

소켓 만 개를 스레드 하나로

동시 접속

동시 접속 만 개 = 소켓 만 개

소켓마다 스레드

소켓마다 기다리는 스레드 → 스레드 만 개

운영체제에 맡김

운영체제에 소켓 목록을 맡기고, 데이터가 온 것만 받아 오기

© 한상훈 NestJS 백엔드 마스터

epoll

→ 리눅스 커널 기능

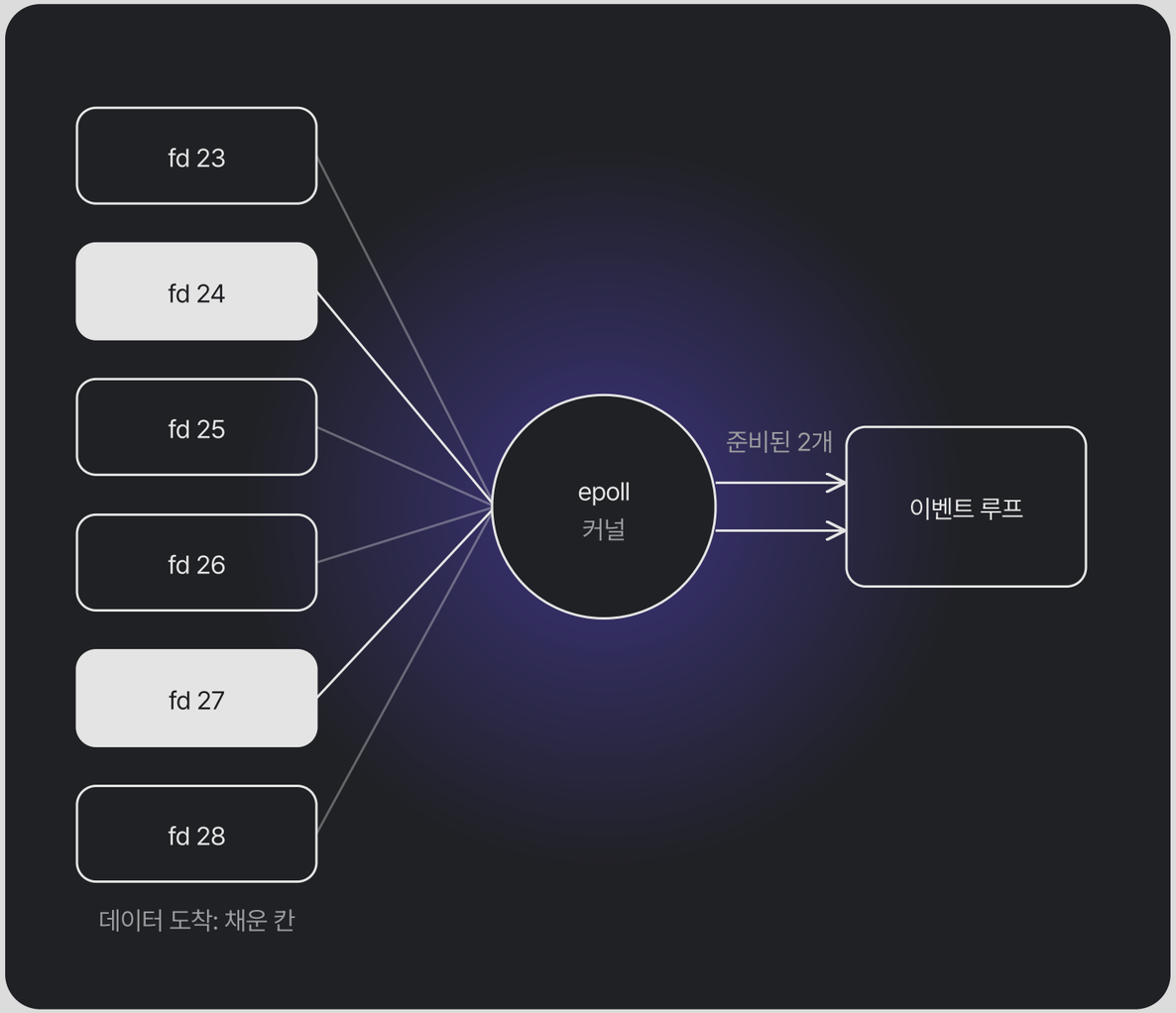
→ 등록: 지켜볼 소켓을 epoll에 추가

→ 대기: "등록한 것 중 준비된 게 생기면 깨워 줘"

→ 깨어나면: 준비된 소켓 목록만 한 번에 받음

→ poll 칸에서 루프가 잠드는 곳 = epoll 대기

→ 옛 방식 select, poll: 깨어날 때마다 전부 확인 → 손님 수만큼 느려짐



© 한상훈 NestJS 백엔드 마스터

서버만 연 상태

터미널

```
$ node epoll-server.mjs
서버만 연 상태: epoll(fd 17)이 지켜보는 소켓 1개 -> fd 23
```

→ `/proc/self/fd`: 프로세스가 연 파일 디스크립터 목록

→ fd 17: `anon_inode:[eventpoll]`, fd 23: 손님을 받는 소켓

데이터 블록: 2026-10-26 WSL2 Ubuntu 24.04, Node v22.22.0 (부록 시트), 촬영 직전 확인

© 한상훈 NestJS 백엔드 마스터

손님 3명 접속 뒤

터미널

```
손님 3명 접속 뒤: epoll(fd 17)이 지켜보는 소켓 4개 -> fd 23, 24, 25, 26
```

→ 손님마다 소켓 하나씩 epoll에 등록

→ 스레드 수: 그대로

→ 손님 만 명 → 등록 만 개, 스레드는 그대로

→ epoll의 일: 데이터 도착 알림까지, 읽기·파싱·컨트롤러 실행은 루프 스레드

© 한상훈 NestJS 백엔드 마스터

운영체제마다 다른 이름

운영체제	쓰는 기능	방식
리눅스	epoll	준비 알림
맥OS	kqueue	준비 알림
윈도우	IOCP	완료 알림

→ 같은 Node 코드, libuv가 운영체제에 맞게 선택

© 한상훈 NestJS 백엔드 마스터

파일은 epoll에 등록 불가

EPERM

디스크의 일반 파일: epoll 등록 시 커널이 거부 (EPERM)

이유

이유: 파일은 늘 "읽을 수 있음"으로 취급, 실제 읽기는 디스크를 기다림

libuv

그래서 libuv: 스레드가 대신 읽고 끝나면 알림

© 한상훈 NestJS 백엔드 마스터

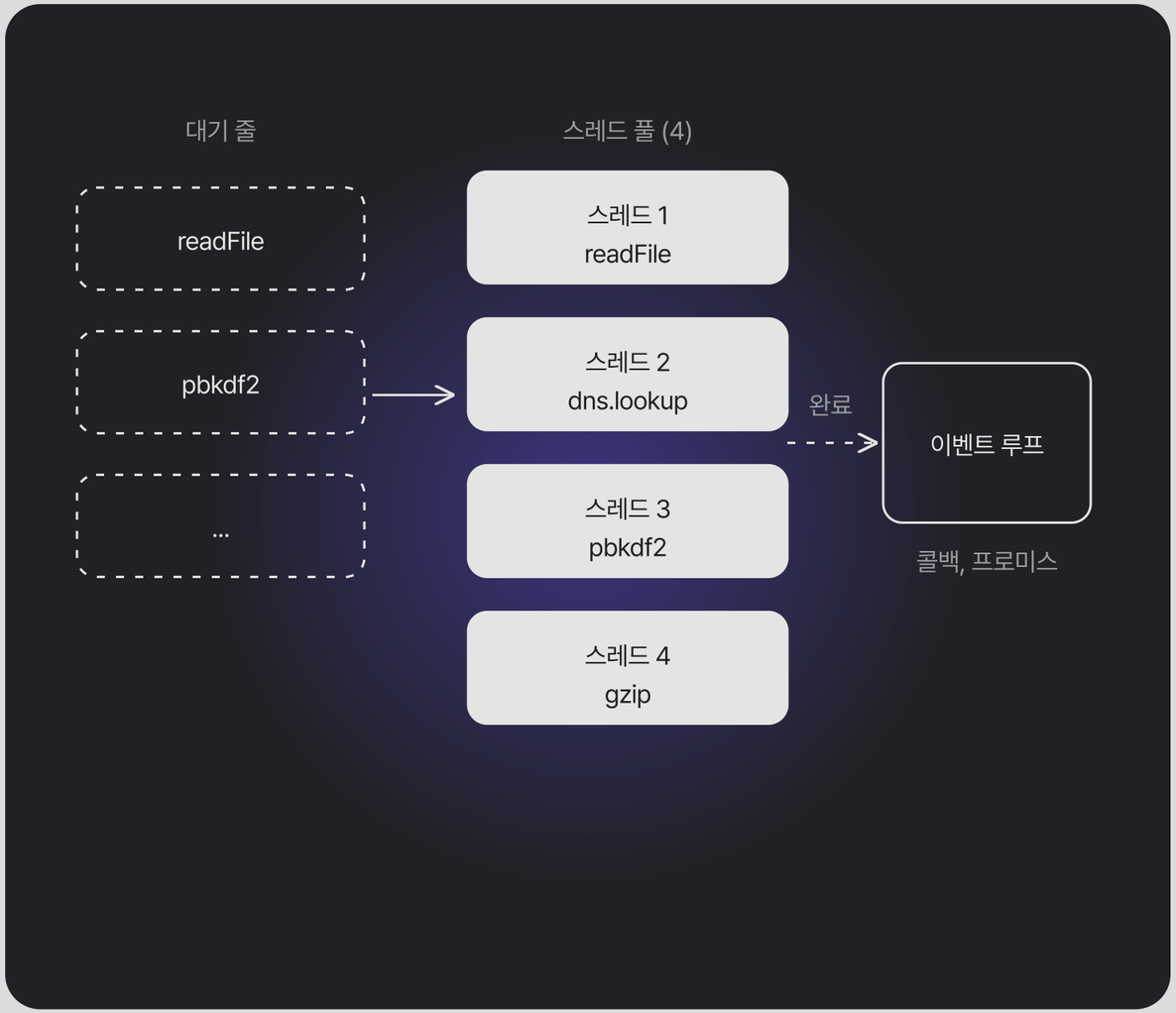
스레드 풀

→ 미리 띄워 둔 스레드 몇 개가 맡긴 일을 차례로 처리

→ libuv 기본 크기: 4

→ 일이 넷보다 많으면: 대기 줄

→ 끝나면: 이벤트 루프에 알림 → 콜백, 프로미스



© 한상훈 NestJS 백엔드 마스터

스레드 풀을 쓰는 API

→ fs의 비동기 API 전부 (`readFile`, `writeFile`, `stat` …)

→ `dns.lookup` (http, fetch가 이름을 찾을 때 기본으로 사용)

→ `crypto.pbkdf2`, `crypto.scrypt`, `crypto.randomBytes` (콜백 버전)

→ `zlib.gzip`, `zlib.deflate` (콜백 버전)

→ 쓰지 않는 것: 소켓 읽기·쓰기, 타이머, `dns.resolve`

→ `dns.resolve`: 풀 미사용, 대신 hosts 파일을 보지 않음

→ NestJS 앱에서: 로그 파일 쓰기, 업로드 저장, gzip 압축, bcrypt 비동기 함수

데이터 블록: Node 공식 문서 "Don't Block the Event Loop" 기준 (부록 시트), 촬영 직전 확인

© 한상훈 NestJS 백엔드 마스터

비동기 해시 8개

pool.mjs

```
import { pbkdf2 } from 'node:crypto';

const n = Number(process.argv[2] ?? 8);
const t0 = performance.now();
for (let i = 1; i <= n; i++) {
  pbkdf2('password', 'salt', 2_000_000, 32, 'sha256', () => {
    console.log(`해시 #${i} 끝: ${(performance.now() - t0).toFixed(0)}ms`);
  });
}
```

→ 콜백 버전 pbkdf2: 스레드 풀에서 계산

© 한상훈 NestJS 백엔드 마스터

네 개씩 두 번

터미널

```
$ node pool.mjs 8
해시 #2 끝: 240ms
해시 #3 끝: 290ms
해시 #4 끝: 293ms
해시 #1 끝: 316ms
해시 #5 끝: 461ms
해시 #8 끝: 543ms
해시 #6 끝: 585ms
해시 #7 끝: 605ms
전체: 605ms
```

→ 첫 넷: 약 0.3초, 다음 넷: 약 0.6초

→ 끝나는 순서 ≠ 부른 순서 (스레드가 코어를 나눠 받는 순서)

데이터 블록: 2026-10-26 강사 PC 실측, Node v24.11.0 (부록 시트), 촬영 직전 확인

© 한상훈 NestJS 백엔드 마스터

UV_THREADPOOL_SIZE=8

터미널

```
$ UV_THREADPOOL_SIZE=8 node pool.mjs 8
해시 #7 끝: 261ms
해시 #4 끝: 264ms
...
해시 #8 끝: 330ms
전체: 330ms
```

→ 여덟 개가 한 번에, 전체 605ms → 330ms

→ 이 PC의 논리 코어 16개

© 한상훈 NestJS 백엔드 마스터

해시가 도는 동안의 dns.lookup

dns.mjs

```
import { pbkdf2 } from 'node:crypto';
import { lookup } from 'node:dns';
import { readFile } from 'node:fs';

const hashes = Number(process.argv[2] ?? 0);
for (let i = 0; i < hashes; i++)
  pbkdf2('password', 'salt', 2_000_000, 32, 'sha256', () => {});

const t0 = performance.now();
lookup('localhost', () => console.log(`dns.lookup: ${(performance.now() - t0).toFixed(0)}ms`));
readFile(import.meta.filename, () => console.log(`readFile: ${(performance.now() - t0).toFixed(0)}ms`));
```

© 한상훈 NestJS 백엔드 마스터

12ms, 203ms, 504ms

같이 도는 해시	풀 크기	dns.lookup	readFile
0개	4	12ms	4ms
4개	4	203ms	204ms
8개	4	504ms	514ms
4개	8	5ms	3ms

데이터 블록: 2026-10-26 강사 PC 실측 (부록 시트), 촬영 직전 확인

© 한상훈 NestJS 백엔드 마스터

로그인이 몰린 날의 결제사 호출

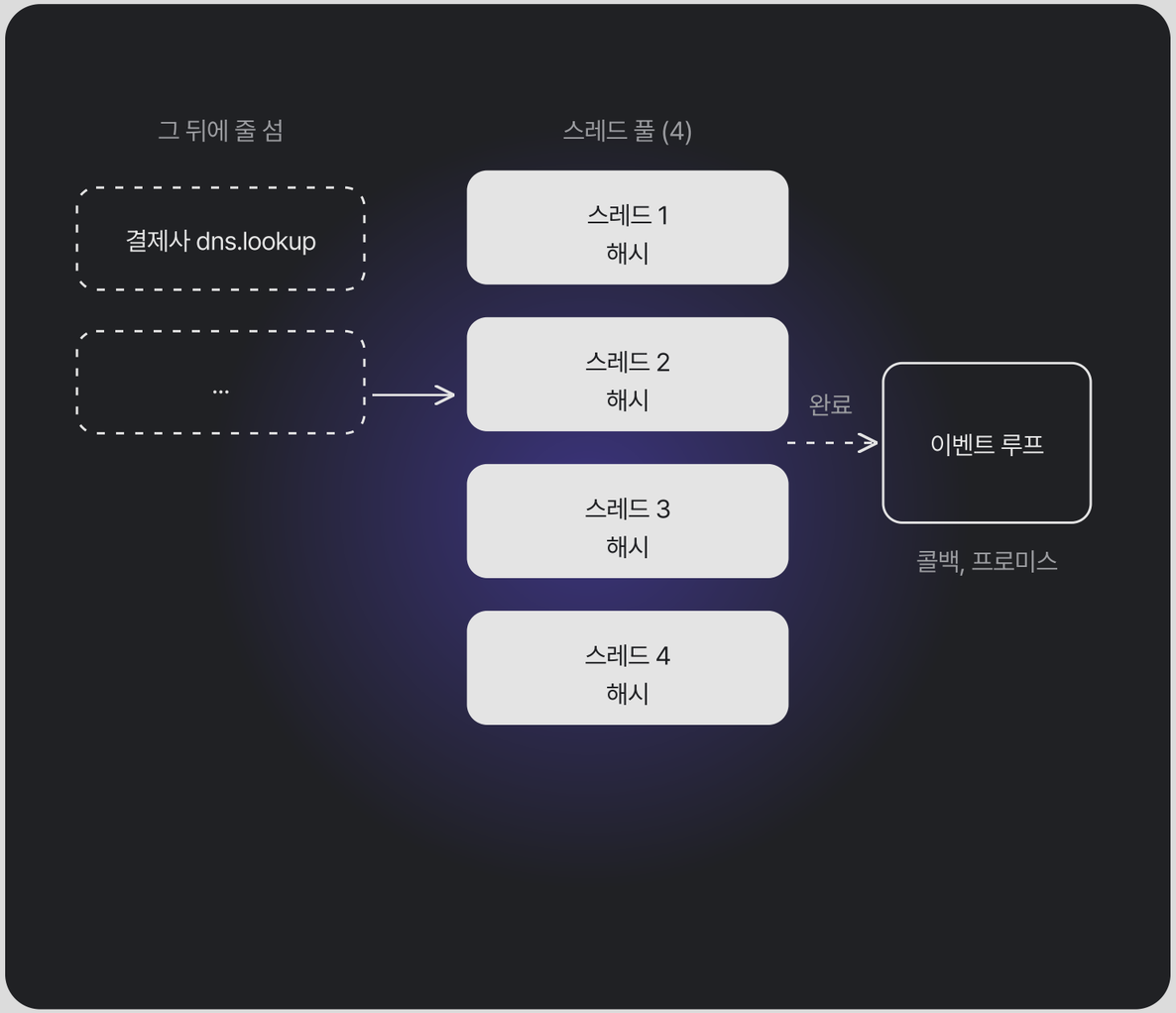
→ 오픈 직전: 로그인 몰림 → 비밀번호 해시가 스레드 풀 차지

→ 같은 서버의 결제 요청: `fetch` → `dns.lookup` → 풀 대기

→ 결제사는 정상, 우리 서버 안에서 이름 찾기 대기

→ 타임아웃 → "결제사 장애" 오인

→ 찾는 법: 외부 호출을 이름 찾기, 연결, 첫 바이트로 나눠 측정



© 한상훈 NestJS 백엔드 마스터

풀 크기 바꾸기

→ 프로세스 시작 때 환경 변수: `UV_THREADPOOL_SIZE=8 node dist/main`

→ 풀은 처음 쓰일 때 한 번 생성, 이후 크기 고정

→ 늘린 스레드도 CPU 코어를 나눠 씀

→ 해시처럼 CPU 쓰는 일: 코어 수 이상으로 늘려도 빨라지지 않음

→ 먼저 확인: 풀을 오래 쓰는 일이 무엇인지

어림셈 예

로그인 초당 20건 × 해시 0.1초 = 스레드 2개 상시 사용, 남는 스레드 2개

© 한상훈 NestJS 백엔드 마스터

준비 알림과 완료 알림

방식	맡기는 말	읽기는 누가
준비 알림 (epoll, kqueue)	"읽을 게 생기면 알려 줘"	알림을 받은 뒤 내가 읽음
완료 알림 (io_uring, IOCP)	"이걸 읽어서 여기 넣어 놓고 알려 줘"	커널이 읽어 둠

© 한상훈 NestJS 백엔드 마스터

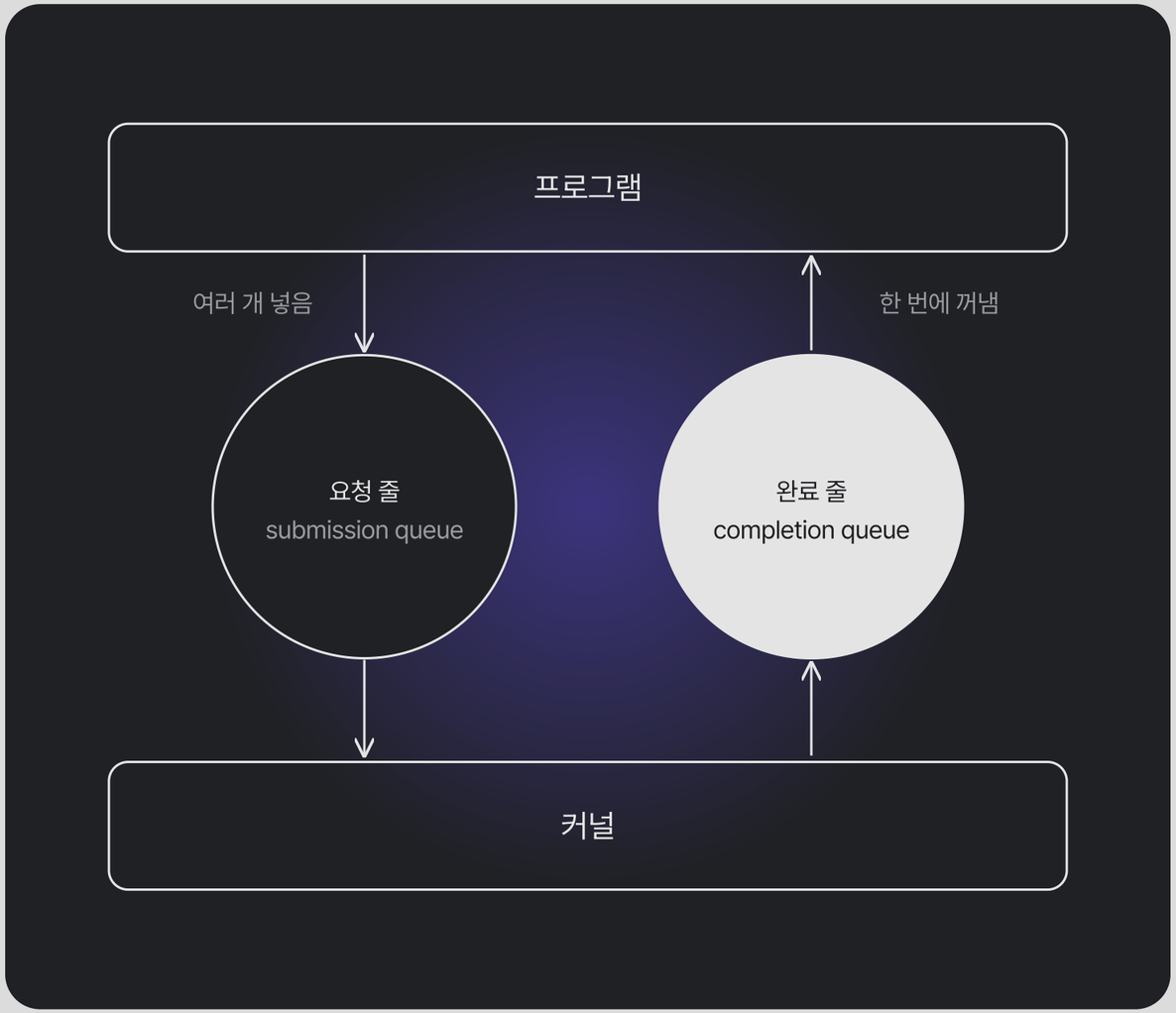
io_uring

→ 리눅스 커널의 비동기 I/O 기능 (커널 5.1부터)

→ 요청 줄(submission queue)과 완료 줄(completion queue)을 커널과 같이 사용

→ 요청 여러 개를 한 번에 넣고, 끝난 것을 한 번에 꺼냄

→ 파일 읽기도 완료 알림으로 → 스레드 풀 없이 가능



© 한상훈 NestJS 백엔드 마스터

Node의 io_uring 기본값

libuv

리눅스에서 io_uring을 쓰는 코드 있음

Node

보안 문제로 기본 꺼짐

환경 변수

켜는 환경 변수:
UV_USE_IO_URING

데이터 블록: Node 20.11.1 보안 릴리스부터 기본 꺼짐, CVE-2024-22017 (부록 시트), 촬영 직전 확인

© 한상훈 NestJS 백엔드 마스터

해시 4개 중 fs.read

터미널

```
node v22.22.0, kernel 6.6.87.2-microsoft-standard-WSL2
기본값
해시 4개 도는 중 fs.read: 227ms
UV_USE_IO_URING=0
해시 4개 도는 중 fs.read: 367ms
UV_USE_IO_URING=1
해시 4개 도는 중 fs.read: 370ms
```

→ 세 경우 모두 해시 뒤에 대기 → 파일 읽기는 스레드 풀

→ 기본값 227ms: 첫 해시가 일찍 끝나 스레드가 빈 순간 읽음, 대기는 동일

데이터 블록: 2026-10-26 WSL2 실측 (부록 시트), 촬영 직전 확인

© 한상훈 NestJS 백엔드 마스터

진동벨 세 종류

길	맡는 일	막히는 때
준비 알림 (epoll, kqueue)	소켓: HTTP, DB 연결, 결제사 연결	거의 없음, 등록 수만큼 메모리
스레드 풀 (기본 4개)	파일, `dns.lookup`, 해시, 압축	넷이 다 찼을 때, 서로 상관없는 일끼리 대기
완료 알림 (io_uring, IOCP)	윈도우의 I/O, 리눅스에서는 켜진 경우	버전에 따라 다름

© 한상훈 NestJS 백엔드 마스터

맡긴 일이 가는 길

"소켓은 운영체제가 지켜보고, 파일과 DNS와
암호 계산은 스레드 네 개가 나눠서 기다립니다."